

Installationsanleitung arbeitszeit

Version: 1.6 **Stand:** Juli 2026 **Zielgruppe:** Laien ohne Linux- oder Programmiererfahrung **Projekt:** Lokales Zeiterfassungssystem für eine Zahnarztpraxis

Für wen ist diese Anleitung?

Diese Anleitung richtet sich an Personen, die das System **arbeitszeit** auf einem Rechner in der Praxis **installieren** sollen, aber keine oder nur wenig Erfahrung mit Linux, der Kommandozeile oder Programmierung haben. Jeder Schritt wird einzeln erklärt, inklusive dem, was der jeweilige Befehl bewirkt und woran man erkennt, dass er erfolgreich war.

Wenn an einer Stelle etwas nicht funktioniert, ist das kein Grund zur Sorge: Am Ende dieser Anleitung findest du einen Abschnitt mit den häufigsten Fehlern und ihren Lösungen.

Was du am Ende dieser Anleitung hast

Nach dem vollständigen Durchlaufen dieser Anleitung ist auf dem Rechner ein lauffähiges Zeiterfassungssystem eingerichtet, mit dem Mitarbeitende per RFID-Karte und Numpad ihre Arbeitszeiten buchen können, und mit dem eine verantwortliche Person Mitarbeiterdaten, Karten, Berichte und Sicherungen verwalten kann.

Außerdem ist eine lokale Konfigurationsdatei `config.toml` eingerichtet, über die Datenbankpfad, Gerätezuordnung, Backup- und Exportverzeichnisse sowie das Log-Verzeichnis zentral verwaltet werden.

Was du vorher wissen solltest

Was ist die Kommandozeile (Terminal)?

Die Kommandozeile (auch „Terminal“ genannt) ist ein Fenster, in das man Textbefehle eintippt, statt mit der Maus zu klicken. Das mag zunächst ungewohnt wirken, ist aber bei diesem System notwendig, weil es bewusst schlank und ohne grafische Oberfläche gebaut wurde. Jeder Befehl in dieser Anleitung wird in einem grauen Kasten dargestellt und kann durch Markieren und Kopieren übernommen werden.

Das Terminal öffnest du unter Linux Mint oder Ubuntu meist über die Tastenkombination `Strg + Alt + T`, alternativ über das Anwendungsmenü (Suche nach „Terminal“).

Was bedeutet sudo?

`sudo` steht für „Superuser do“ und erlaubt es, einen Befehl mit erweiterten Rechten auszuführen. Nach der Eingabe eines Befehls mit `sudo` wirst du nach deinem Benutzerpasswort gefragt. Beim Tippen des Passworts erscheinen aus Sicherheitsgründen keine Zeichen auf dem Bildschirm — das ist normal.

Zeilen mit \$ oder

In manchen Anleitungen im Internet stehen vor Befehlen Zeichen wie `$` oder `#`. Diese Zeichen gehören **nicht** zum Befehl und dürfen nicht mitkopiert werden. In dieser Anleitung wurden solche Zeichen bewusst weggelassen — du kannst jeden Befehl in den grauen Kästen exakt so kopieren, wie er dort steht.

Was du vorher benötigst

- Einen Rechner mit **Linux Mint** oder **Lubuntu**, bereits installiert und einsatzbereit.
 - **LUKS-Festplattenverschlüsselung aktiviert** — zwingend erforderlich (Details und Prüfung: Schritt 0).
 - Eine Internetverbindung — auch nach der Installation dauerhaft benötigt, damit die interne Uhrzeit per NTP synchronisiert bleibt.
 - Administratorrechte auf dem Rechner (also ein Benutzerkonto, mit dem sudo-Befehle funktionieren).
 - Ausreichend Zeit — plane für die komplette Ersteinrichtung etwa 30 bis 60 Minuten ein.
 - Für den Betrieb notwendig: ein RFID-Kartenlesegerät und ein USB-Numpad, beide über USB angeschlossen.
-

Schritt 0: LUKS-Festplattenverschlüsselung sicherstellen

Warum ist das zwingend?

arbeitszeit verarbeitet personenbezogene Daten von Mitarbeitenden. Ohne Festplattenverschlüsselung wären diese Daten bei Diebstahl oder Verlust des Rechners ungeschützt lesbar.

LUKS (Linux Unified Key Setup) ist der Standardmechanismus zur Festplattenverschlüsselung unter Linux. Die Verschlüsselung muss **vor** der Installation von **arbeitszeit** aktiv sein — eine nachträgliche Einrichtung ist praktisch nur über eine Neuinstallation sauber möglich.

Prüfen, ob LUKS bereits aktiv ist

```
lsblk -o NAME,TYPE,FSTYPE
```

In der Ausgabe nach Einträgen vom Typ `crypt` suchen:

NAME	TYPE	FSTYPE
sda	disk	
└─sda1	part	crypto_LUKS
└─dm-0	crypt	ext4

Erscheint in der Spalte TYPE der Wert `crypt`, ist LUKS aktiv — du kannst mit Schritt 1 fortfahren.

Erscheint **kein** Eintrag vom Typ `crypt`, ist die Festplatte **nicht** verschlüsselt. Das System muss **neu installiert** werden, bevor **arbeitszeit** eingerichtet werden darf.

LUKS bei der Neuinstallation aktivieren

Beim Installieren von Linux Mint oder Lubuntu:

1. Im Installationsschritt „Installationsart“ die Option **„Festplatte löschen und Linux Mint installieren“** (bzw. „... Lubuntu installieren“) wählen.
2. Den Haken setzen bei **„Das neue System zur Sicherheit verschlüsseln“**.
3. Ein **starkes Verschlüsselungspasswort** festlegen und sicher verwahren. Dieses Passwort wird bei jedem Systemstart abgefragt.

Hinweis: Das Verschlüsselungspasswort für LUKS ist **unabhängig** vom Benutzerpasswort des Betriebssystems.

Schritt 1: System aktuell halten

Bevor irgendetwas installiert wird, sollte das Betriebssystem auf den neuesten Stand gebracht werden:

```
sudo apt update
sudo apt upgrade -y
```

Der erste Befehl aktualisiert die Paketlisten. Der zweite Befehl installiert alle verfügbaren Aktualisierungen.

Schritt 2: Python 3.14 installieren

Das System `arbeitszeit` benötigt **Python 3.14**. Das ist im Projekt explizit festgelegt: `requires-python = ">=3.14,<3.15"`. Keine andere Python-Version funktioniert.

Prüfen, ob Python 3.14 bereits vorhanden ist:

```
python3.14 --version
```

Falls eine Ausgabe wie `Python 3.14.x` erscheint, ist dieser Schritt bereits erledigt. Erscheint stattdessen eine Fehlermeldung wie `command not found`, muss Python 3.14 nachinstalliert werden.

Auf manchen Linux-Mint- und Ubuntu-Versionen ist Python 3.14 noch nicht im Standard-Paketverzeichnis enthalten. Prüfe zunächst mit dem normalen Installationsbefehl:

```
sudo apt install -y python3.14 python3.14-venv python3-pip
```

Schlägt dieser Befehl mit einer Meldung wie `Paket python3.14 nicht gefunden` fehl, füge zuerst das Paketarchiv „deadsnakes“ hinzu und wiederhole dann:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.14 python3.14-venv python3-pip
```

Anschließend die Version erneut prüfen:

```
python3.14 --version
```

Die Ausgabe muss `Python 3.14.x` zeigen.

Schritt 3: Zusätzliche Systempakete installieren

Einige Teile von `arbeitszeit` benötigen zusätzliche Pakete, insbesondere für `evdev` und die Geräteprüfung:

```
sudo apt install -y build-essential linux-headers-$(uname -r) python3-dev evtest
```

Diese Pakete werden für das Bauen und Nutzen der Hardware-Anbindung sowie für die Geräteidentifikation mit `evtest` benötigt. Das Python-Projekt selbst deklariert `evdev` als feste Laufzeitabhängigkeit.

Schritt 4: Programmcode herunterladen

Wechsle zunächst in ein Verzeichnis, in dem das Projekt abgelegt werden soll, zum Beispiel dein Home-Verzeichnis:

```
cd ~
```

Falls `git` noch nicht installiert ist:

```
sudo apt install -y git
```

Lade nun den Programmcode herunter:

```
git clone https://github.com/iCodator/arbeitszeit.git
```

Danach in den Projektordner wechseln:

```
cd arbeitszeit
```

Schritt 5: Virtuelle Umgebung einrichten

Virtuelle Umgebung anlegen:

```
python3.14 -m venv .venv
```

Virtuelle Umgebung aktivieren:

```
source .venv/bin/activate
```

Nach dem Aktivieren erscheint vor der Eingabeaufforderung im Terminal die Kennzeichnung (.venv).

Wichtig: Bei jeder neuen Terminal-Sitzung muss die virtuelle Umgebung erneut aktiviert werden:

```
source .venv/bin/activate
```

Ohne diesen Befehl erscheinen Fehlermeldungen wie `ModuleNotFoundError`. Die Aktivierung entfällt erst dann, wenn ein `systemd`-Service eingerichtet wurde.

Schritt 6: Abhängigkeiten installieren

Jetzt werden die Python-Abhängigkeiten des Projekts installiert:

```
pip install -e .[dev]
```

Dieser Befehl liest `pyproject.toml` und installiert sowohl die Projektabhängigkeiten als auch die im `dev`-Extra definierten Test- und Entwicklungswerkzeuge. Dazu gehören unter anderem `evdev`, `reportlab`, `pytest`, `pytest-cov`, `pypdf`, `ruff` und `import-linter`.

Schritt 7: Datenbank anlegen

`arbeitszeit` verwendet eine lokale SQLite-Datenbank. Beim ersten Start müssen die Migrationen ausgeführt werden:

```
python scripts/init_db.py
```

`init_db.py` legt dabei standardmäßig die Datei `arbeitszeit.db` an, führt alle vorhandenen Migrationen aus und gibt jede erfolgreich angewendete Migration einzeln aus. Im Repository sind aktuell die Migrationen 0001 bis 0006 enthalten.

Eine typische Ausgabe sieht so aus:

```
Migration 0001 angewendet.  
Migration 0002 angewendet.  
Migration 0003 angewendet.  
Migration 0004 angewendet.  
Migration 0005 angewendet.  
Migration 0006 angewendet.
```

△ Ersteinrichtung noch erforderlich:
`python scripts/setup.py --db arbeitszeit.db`

Das △-Symbol bedeutet hier **keinen Fehler**, sondern den Hinweis, dass danach noch die Konfiguration erstellt werden muss.

Schritt 8: Ersteinrichtung und `config.toml` erstellen

Dieser Schritt erstellt und pflegt die zentrale Konfigurationsdatei `config.toml`. Das Script `scripts/setup.py` fragt alle benötigten Werte interaktiv ab und speichert sie in dieser Datei.

Einfacher interaktiver Start:

```
python scripts/setup.py --db arbeitszeit.db
```

Dabei wird eine `config.toml` geschrieben. Der Schreibpfad wird automatisch ermittelt: bevorzugt eine bereits vorhandene Konfigurationsdatei, sonst der XDG-Standardpfad `~/.config/arbeitszeit/config.toml`.

Welche Werte werden eingerichtet?

`setup.py` unterstützt unter anderem diese Einstellungen:

- Datenbankpfad (`--db` beziehungsweise `database.path` in der Konfiguration).
- Terminal-ID (`--terminal-id`).
- Gerätenamen für das Numpad (`--numpad`).
- Gerätenamen für den RFID-Reader (`--rfid`).
- Benutzer-ID eines Administrators (`--admin-user-id`).
- Backup-Verzeichnis (`--backup-dir`).
- Exportverzeichnis (`--export-dir`).
- Log-Verzeichnis (`--log-dir`).

Beispiel für einen nicht-interaktiven Aufruf

```
python scripts/setup.py \
  --db arbeitszeit.db \
  --terminal-id 1 \
  --numpad "USB Numpad" \
  --rfid "HID 1234:5678" \
  --admin-user-id 1 \
  --backup-dir /var/backups/arbeitszeit \
  --export-dir /var/exports/arbeitszeit \
  --log-dir /var/log/arbeitszeit
```

Die Gerätenamen für `--numpad` und `--rfid` können in diesem Schritt noch leer gelassen und in Schritt 9 nachgetragen werden.

Was zeigt das Script am Ende an?

Nach erfolgreicher Ausführung gibt `setup.py` Startbeispiele für die Terminal-UI und die Admin-CLI aus.

Schritt 9: Zugriff auf RFID-Reader und Numpad einrichten

RFID-Reader und Numpad erscheinen unter Linux als Eingabegeräte. Um ihre Gerätenamen zu ermitteln, führe aus:

```
sudo evtest
```

Beispielausgabe:

```
/dev/input/event0:  Power Button
/dev/input/event3:  USB Numpad
/dev/input/event4:  HID 1234:5678
```

Wichtig ist vor allem der **Gerätename** nach dem Doppelpunkt, zum Beispiel USB Numpad oder HID 1234:5678. Das Projekt unterstützt stabile Gerätenamen als Eingabe für `--numpad` und `--rfid`; zusätzlich werden auch direkte Gerätepfade akzeptiert.

Achtung: Gerätenamen exakt übernehmen. Der Gerätename muss buchstabengenaue mit der Ausgabe von `sudo evtest` übereinstimmen, einschließlich Groß- und Kleinschreibung und Sonderzeichen. Ein einziger Unterschied verhindert den Start des Systems. Verwende daher immer **Kopieren und Einfügen** statt manuelles Abtippen.

Damit dein Benutzerkonto die Eingabegeräte lesen darf, zur Gruppe `input` hinzufügen:

```
sudo usermod -aG input $USER
```

Unbedingt beachten: Jetzt abmelden und neu anmelden.

Die Gruppenmitgliedschaft wird **erst nach einer vollständigen Abmeldung** wirksam. Ein reines Schließen des Terminals reicht nicht aus — du musst dich am Betriebssystem komplett abmelden und dann wieder anmelden (oder den Rechner neu starten).

Ohne diesen Schritt schlagen alle folgenden Hardware-Tests mit der Meldung `Permission denied` fehl.

Wenn du dich wieder angemeldet und das Terminal erneut geöffnet hast, wechsele wieder in den Projektordner und aktiviere die virtuelle Umgebung:

```
cd ~/arbeitszeit
source .venv/bin/activate
```

Trage jetzt die ermittelten Gerätenamen in `config.toml` ein, falls du sie in Schritt 8 noch nicht angegeben hast:

```
python scripts/setup.py \
  --numpad "Gerätename Numpad" \
  --rfid "Gerätename RFID-Reader"
```

Schritt 10: Hardware testen

Bevor das erste Benutzerkonto angelegt wird, solltest du prüfen, ob Numpad und RFID-Reader korrekt funktionieren. Dafür gibt es das Script `scripts/verify_hardware.py`.

Zuerst die Gerätepfade mit `sudo evtest` oder anhand der Ausgabe von Schritt 9 ermitteln (zum Beispiel `/dev/input/event3` und `/dev/input/event4`):

```
python scripts/verify_hardware.py \
  --numpad /dev/input/event3 \
  --rfid /dev/input/event4
```

Das Script führt drei Überprüfungen durch:

1. **Gerätezugriff:** Sind die Dateien `/dev/input/eventX` vorhanden und lesbar?

2. **Numpad:** Drücke innerhalb von 15 Sekunden eine Taste auf dem Numpad (1 = Kommen, 2 = Gehen, 3 = Pause Start, 4 = Pause Ende).
3. **RFID-Reader:** Halte innerhalb von 15 Sekunden eine RFID-Karte an den Leser.

Wenn alles funktioniert, zeigt das Script für jeden Test OK an. Bei einem Fehler erscheint eine beschreibende Fehlermeldung.

Das Script gibt außerdem den **SHA-256-Hash** der eingelesenen RFID-Karte aus. Dieser Hash wird in der Datenbank gespeichert und kann später beim Zuweisen von Karten verwendet werden.

Um nur die Gerädateien zu prüfen (ohne interaktive Tasteneingabe):

```
python scripts/verify_hardware.py \  
  --numpad /dev/input/event3 \  
  --rfid /dev/input/event4 \  
  --skip-interactive
```

Die konkreten Gerätenamen dieser Installation sind in der Datei docs/03_installation_technik/hardware.md festgehalten.

Schritt 11: Backup-Verzeichnisse anlegen

Beim Start des Systems prüft arbeitszeit automatisch, ob die in config.toml eingetragenen Verzeichnisse für Backups und Exporte tatsächlich auf der Festplatte vorhanden sind. Fehlen diese Verzeichnisse, meldet der Selbsttest einen Fehler.

Lege die Verzeichnisse jetzt an und gib deinem Benutzerkonto Schreibzugriff:

```
sudo mkdir -p /var/backups/arbeitszeit  
sudo mkdir -p /var/exports/arbeitszeit  
sudo mkdir -p /var/log/arbeitszeit  
sudo chown $USER /var/backups/arbeitszeit  
sudo chown $USER /var/exports/arbeitszeit  
sudo chown $USER /var/log/arbeitszeit
```

Falls du in Schritt 8 andere Verzeichnispfade gewählt hast, passe die obigen Befehle entsprechend an.

Schritt 12: Erstes Administratorkonto anlegen

Bevor das System genutzt werden kann, muss ein erstes Administrator-Benutzerkonto angelegt werden.

Direkter Aufruf mit Datenbankpfad

```
python -m arbeitszeit.presentation.admin_cli.main \  
  --db arbeitszeit.db \  
  users bootstrap \  
  --username adminname
```

Der Bootstrap-Befehl ist der einzige Befehl der Admin-Verwaltung, der keine vorherige --user-id benötigt, da noch kein Administrator existiert.

Alternativ über die Konfigurationsdatei

Wenn Schritt 8 bereits eine config.toml geschrieben hat und darin der Datenbankpfad gesetzt ist, kann stattdessen auch mit --config gearbeitet werden:

```
python -m arbeitszeit.presentation.admin_cli.main \
  --config ~/.config/arbeitszeit/config.toml \
  users bootstrap \
  --username adminname
```

Bei Erfolg erscheint eine Ausgabe in dieser Form:

Erstes Administratorkonto angelegt (ID 1).
Generiertes Passwort (einmalig sichtbar): <zufälliges Passwort>

Unbedingt beachten: Das Passwort sofort notieren.

Das angezeigte Passwort wird **genau einmal** angezeigt und kann danach nicht mehr abgerufen werden. Notiere es jetzt sofort und verwahre es sicher — zum Beispiel in einem verschlossenen Umschlag oder einem Passwortmanager.

Ohne dieses Passwort ist kein Zugang zur Admin-Verwaltung möglich.

Schritt 13: Mitarbeitende und Karten anlegen

Nach dem Administrator-Konto können weitere Benutzer, Mitarbeitende und RFID-Karten angelegt werden.

Weiteres Benutzerkonto anlegen

```
python -m arbeitszeit.presentation.admin_cli.main \
  --db arbeitszeit.db \
  --user-id 1 \
  users add \
  --username pruefer01 \
  --role REVIEWER
```

Die Rollen ADMIN, REVIEWER und TECH sind als erlaubte Werte für `users add` registriert.

Mitarbeiter anlegen

```
python -m arbeitszeit.presentation.admin_cli.main \
  --db arbeitszeit.db \
  --user-id 1 \
  employees add \
  --personnel-no M001 \
  --first-name Maria \
  --last-name Mustermann
```

RFID-Karte direkt einlesen und zuweisen

```
python -m arbeitszeit.presentation.admin_cli.main \
  --db arbeitszeit.db \
  --user-id 1 \
  cards assign \
  --employee-id 1 \
  --scan \
  --rfid "HID 1234:5678"
```


Danach erscheint die Meldung Bitte Karte an den RFID-Reader halten Bei erfolgreicher Zuordnung erscheint:

Karte zugewiesen (ID 1).

Hinweis zur Nutzung von --config

Auch alle oben gezeigten Admin-CLI-Aufrufe können statt mit --db mit --config arbeiten, sofern die Konfigurationsdatei bereits vollständig vorliegt:

```
python -m arbeitszeit.presentation.admin_cli.main \
  --config ~/.config/arbeitszeit/config.toml \
  --user-id 1 \
  employees add \
  --personnel-no M001 \
  --first-name Maria \
  --last-name Mustermann
```

Schritt 14: Funktionstest durchführen

Zum Prüfen der Installation können die automatisierten Tests ausgeführt werden:

pytest

Die Projektkonfiguration legt tests als Testverzeichnis fest.

Wenn nur gezielt ein Teil geprüft werden soll, können einzelne Testdateien unter tests/ ausgeführt werden. Der genaue Dateiname sollte dabei vorab im Repository geprüft werden.

Schritt 15: Terminal-Betrieb starten

Die Terminal-UI unterstützt zwei Wege: direkte Übergabe aller Werte oder Start über die zentrale config.toml. Der Code priorisiert dabei CLI-Werte vor Konfigurationswerten.

Direkter Start mit allen Parametern

```
python -m arbeitszeit.presentation.terminal_ui.main \
  --db arbeitszeit.db \
  --numpad "USB Numpad" \
  --rfid "HID 1234:5678" \
  --terminal-id 1
```

Empfohlener Start über config.toml

Wenn Schritt 8 bereits alle Werte geschrieben hat, genügt in der Regel:

```
python -m arbeitszeit.presentation.terminal_ui.main \
  --config ~/.config/arbeitszeit/config.toml
```

Die Terminal-UI liest dann database.path, terminal.numpad, terminal.rfid und terminal.id aus der Konfiguration.

Gerätenamen statt Gerätepfade

Für `--numpad` und `--rfid` sind stabile Gerätenamen ausdrücklich vorgesehen. Vor dem Start werden diese Namen intern in konkrete Gerätepfade aufgelöst.

Wie wird der Betrieb beendet?

Mit `Strg + C` im Terminal. Die Terminal-UI behandelt `SIGINT` und `SIGTERM` ausdrücklich kontrolliert.

Häufige Probleme und Lösungen

Problem	Mögliche Ursache	Lösung
<code>command not found:</code> <code>python3.14</code> Paket <code>python3.14</code> nicht gefunden beim <code>apt install</code>	Python 3.14 ist nicht installiert Python 3.14 noch nicht im Standard-Paketarchiv	Schritt 2 wiederholen; ggf. das <code>deadsnakes-PPA</code> hinzufügen. <code>sudo add-apt-repository ppa:deadsnakes/ppa</code> && <code>sudo apt update</code> ausführen, dann erneut installieren.
Permission denied beim Hardware-Test	Benutzerkonto nicht in Gruppe <code>input</code> oder Neuanmeldung fehlt	Schritt 9 wiederholen, dann vollständig abmelden und neu anmelden.
Terminal-UI startet nicht: Gerät nicht gefunden	Gerätename ist falsch oder das Gerät ist nicht angeschlossen	Schritt 9 erneut ausführen und den Gerätenamen exakt per Kopieren übernehmen.
<code>ModuleNotFoundError</code> bei Programmstart	Virtuelle Umgebung nicht aktiv	Im Projektordner <code>source .venv/bin/activate</code> ausführen.
Fehler: DB-Pfad erforderlich in der Admin-CLI Fehler: <code>--scan</code> erfordert <code>--rfid</code>	Weder <code>--db</code> noch eine passende <code>config.toml</code> ist vorhanden Beim Karten-Scan fehlt die Reader-Angabe	<code>--db</code> angeben oder Schritt 8 korrekt abschließen. <code>cards assign</code> mit <code>--scan --rfid "<Gerätename>"</code> ausführen.
Terminal-UI meldet fehlende Konfiguration	<code>--config</code> verwendet, aber <code>config.toml</code> enthält nicht alle Pflichtwerte	Schritt 8 erneut ausführen und fehlende Werte ergänzen.
Selbsttest meldet <code>SELFTEST_FAIL: config_file_paths</code>	Backup- oder Exportverzeichnis existiert nicht	Schritt 11 ausführen und fehlende Verzeichnisse anlegen.
Selbsttest meldet <code>SELFTEST_FAIL: ntp_sync</code>	Keine Internetverbindung oder NTP-Dienst nicht aktiv	Internetverbindung prüfen; <code>timedatectl status</code> im Terminal ausführen.

Kurzglossar für Einsteiger

- **Terminal / Kommandozeile:** Fenster zur Texteingabe von Befehlen.
- **sudo:** Ausführung eines Befehls mit Administratorrechten.
- **Repository:** Der komplette Programmcode-Ordner eines Projekts.
- **Virtuelle Umgebung:** Ein abgeschlossener Bereich für Python-Pakete.
- **Migration:** Ein automatisierter Schritt, der die Datenbankstruktur anlegt oder erweitert.

- **RFID:** Funktechnik zur kontaktlosen Identifikation.
- **Admin-CLI:** Textbasierte Verwaltungsoberfläche des Systems.
- **Terminal-UI:** Der laufende Buchungsprozess für den Praxisbetrieb.
- **config.toml:** Zentrale Konfigurationsdatei für Datenbank, Geräte, Verzeichnisse und weitere Laufzeitwerte.
- **NTP:** Netzwerkprotokoll zur automatischen Zeitsynchronisation.
- **LUKS:** Linux-Festplattenverschlüsselung zum Schutz persönlicher Daten.

Wo finde ich weitere Informationen?

Weitere technische Dokumentation liegt im Repository unter docs/ in den entsprechenden Themenordnern. Insbesondere:

- docs/03_installation_technik/befehlsreferenz.md — vollständige Referenz aller Admin-CLI-Befehle
- docs/03_installation_technik/hardware.md — konkrete Gerätenamen dieser Installation
- docs/04_betrieb/handbuch_backup_restore.md — Anleitung für Datensicherung und Wiederherstellung
- docs/04_betrieb/backup_zeitplan_und_automatisierung.md — Einrichten automatischer Backups per systemd-Timer oder cron